UNIVERSITÄT PASSAU
Fakultät für Informatik und Mathematik

Dependable Distributed Systems – 5880V/UE

05. Broadcast semantics and algorithms (part 2) – 2026-11-09

Prof. Dr. Hans P. Reiser | WS 2026/27

UNIVERSITÄT PASSAU



Overview

1. Reliable broadcast with Byzantine faults

2. Implementing atomic broadcast
   - Sequencer-based approaches
   - Token-based approaches
   - Communication history-based approaches
   - Consensus-based approaches

3. Consensus
   - Definition
   - Transformations

Reliable broadcast with Byzantine faults

(First) Problem: Faulty sender may send inconsistent values

- Simple echo algorithms (for crashes) makes sure that some value is delivered
- But not that all delivered values are equal
- . . . think about a solution!

Reliable broadcast with Byzantine faults

(First) Problem: Faulty sender may send inconsistent values

- Simple echo algorithms (for crashes) makes sure that some value is delivered
- But not that all delivered values are equal
- . . . think about a solution!

(First) Solution: Collect votes from all processes

- There are at least $n - f$ correct processes
- Accept value v if more than half of all correct processes vote in favour of v
- More than $f + \frac{n-f}{2} = \frac{n+f}{2}$ votes implies votes from more than half of all correct processes (without knowing which processes are correct)

Reliable broadcast with Byzantine faults

Implication (without synchrony):

- f faulty processes might not send anything

Reliable broadcast with Byzantine faults

Implication (without synchrony):

- f faulty processes might not send anything
- Any algorithm should do something after receiving messages from only $n - f$ processes

Reliable broadcast with Byzantine faults

Implication (without synchrony):

- f faulty processes might not send anything
- Any algorithm should do something after receiving messages from only $n - f$ processes
- If sender is correct, a positive vote should always be made, i.e., $n - f > \frac{n+f}{2}$ must hold
- This is equivalent to $n > 3f$

Reliable broadcast with Byzantine faults

Implication (without synchrony):

- f faulty processes might not send anything
- Any algorithm should do something after receiving messages from only $n - f$ processes
- If sender is correct, a positive vote should always be made, i.e., $n - f > \frac{n+f}{2}$ must hold
- This is equivalent to $n > 3f$

For tolerating f Byzantine faults, we (usually) need at least $3f + 1$ processes

Echo broadcast with Byzantine faults

- $n = 3f + 1$ processes (or generically $f = \lfloor (n-1)/3 \rfloor$)
- What does it guarantee?

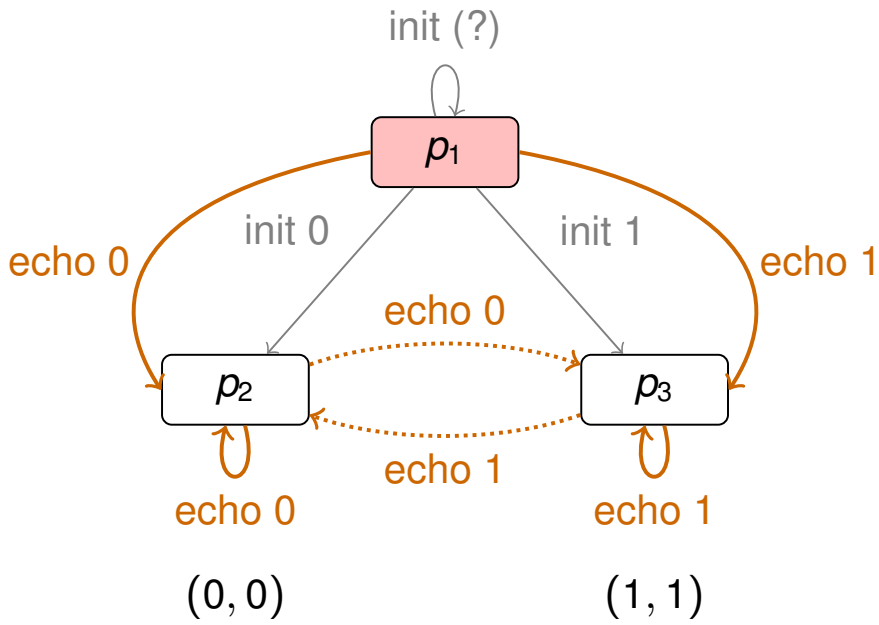
1. G sends (initial, G, m) message to all processes.
2. When a process receives the first (initial, G, m) message from G, it sends an (echo, G, m) message to all processes. All subsequent (initial, G, m') messages are ignored.
3. If a process receives an (echo, G, m) message from more than $\frac{n+f}{2}$ distinct processes, then it accepts the message *m* from G.

(from: Sam Toueg: Randomized Byzantine Agreements, 1984)

Why $n = 3f$ fails

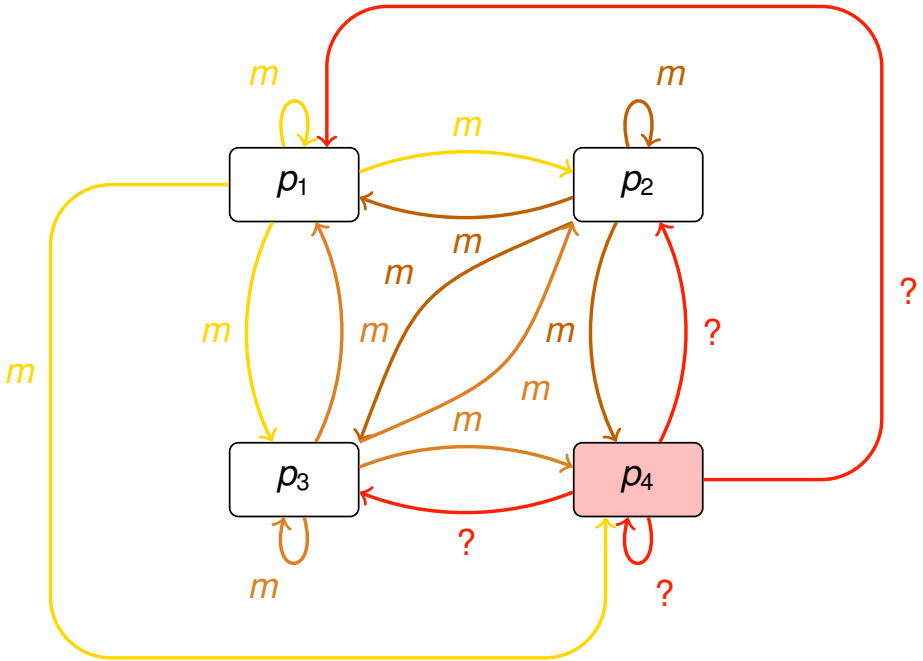
- p_1: initial to all (incl. itself), malicious
- p_2 has $(0, 0)$; p_3 has $(1, 1)$ (threshold 2)
- Inconsistent delivery

- Larger threshold: not a solution (a process can stay silent)



Echo threshold with $n = 4$

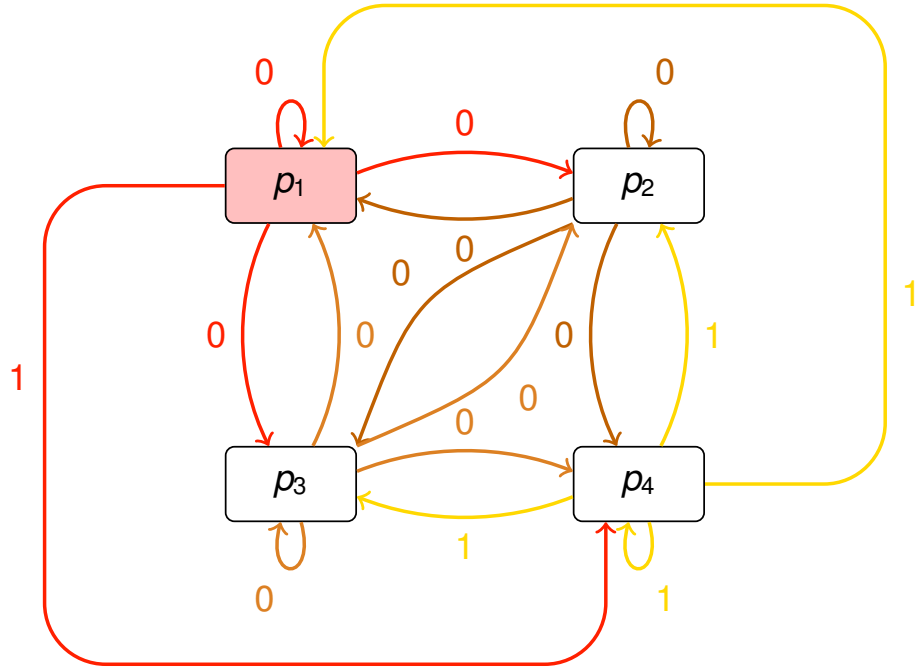
- p_1: initial m to all (incl. itself)
- p_4 malicious: may echo anything (or nothing)
- Threshold 3: correct processes still get (m, m, m) and accept
- Consistent delivery



Echo threshold: malicious broadcaster

- p_1 malicious broadcaster
- p_2, p_3 receive $(0, 0, 0, 1)$ and deliver 0
- p_4 receives $(0, 0, 1, 1)$ and delivers nothing

- Not all-or-nothing



Echo broadcast with Byzantine faults

(in Toueg-Paper: correct = “proper”, deliver = “accept”)

If a process G broadcasts the message *m* with echo broadcast then:

1. The messages delivered by correct processes are identical
2. If G is correct, then all the correct processes deliver *m*

- *Note: Property 1 guarantees only that the delivered messages are identical (i.e., if two messages are delivered, they are the same). If G is faulty, it is possible that some processes deliver m, and others do* ***not*** *deliver m.*
- This is Byzantine best-effort broadcast!

Reliable broadcast with Byzantine faults

For reliable broadcast:

- Requirement: If one correct process delivers some message m, all correct processes must eventually deliver m (even if $sender(m)$ is faulty)

Reliable broadcast with Byzantine faults

For reliable broadcast:

- Requirement: If one correct process delivers some message m, all correct processes must eventually deliver m (even if $sender(m)$ is faulty)

- Algorithm by Bracha 1984
  - Use "more than $\frac{n+f}{2}$ votes" argument to ensure that at most one message can be delivered.
  - Use additional step to guarantee "all-or-nothing" behaviour if sender is faulty.

Reliable broadcast (Bracha 1984)

- $n = 3f + 1$, operation Broadcast(v)

1. (by transmitter only): send (initial, v) to all processes.
2. Wait until receive for some v:
   - one (initial, v) message
   - or $> \frac{n+f}{2}$ (echo, v) messages
   - or $(f + 1)$ (ready, v) messages

   send (echo, v) to all processes
3. Wait until receive for some v:
   - $> \frac{n+f}{2}$ (echo, v) messages
   - or $(f + 1)$ (ready, v) messages

   send (ready, v) to all processes.
4. Wait until receive for some v:
   - $2f + 1$ (ready, v) messages

   accept v

(from: G. Bracha: An asynchronous $\lfloor (n-1)/3 \rfloor$-resilient consensus protocol, 1984)

Reliable broadcast (Bracha 1984): Properties

1. If p is correct, then all the correct processes accept the value of its message.
2. If p is malicious, then either all the correct processes accept the same value, or none of them will accept any value from p.

Reliable broadcast (Bracha 1984): Proof

1. **Lemma 1:** If two correct processes r and s send *(ready,v)* and *(ready,u)* messages, respectively, then $u = v$.
2. **Lemma 2:** If two correct processes p and q accept the values v and u, respectively, then $u = v$.
3. **Lemma 3:** If a correct process p accepts the value v, then every other correct process will eventually accept v.
4. **Lemma 4:** If the transmitter p is correct and it sends v, then all correct processes will accept v.

Lemma 1

- Correct processes only echo one value v
- First (ready, v) from a correct process cannot be based on received (ready, v) messages, so there must be $> \frac{n+f}{2}$ (echo, v) messages
- Then there are $> \frac{n-f}{2}$ (echo, v) from correct processes; those do not vote for a different message
- A second value u would also need $> \frac{n-f}{2}$ (echo, u) from correct processes
- That requires $> \frac{n-f}{2} + > \frac{n-f}{2}$, hence $> n - f$ messages from correct processes, but only $n - f$ correct processes guaranteed to exist

Lemmas 2–4

The arguments for Lemmas 2–4 are an exercise.

Summary of broadcast echo algorithms

Crash faults

- Simple echo

Byzantine faults

- Simple echo + threshold
- “Two-phase echo” + thresholds

Summary of broadcast echo algorithms

Crash faults

- Simple echo for implementing **reliable broadcast**

Byzantine faults

- Simple echo + threshold for implementing **best effort broadcast**
- “Two-phase echo” + thresholds for implementing **reliable broadcast**

Overview

1. Reliable broadcast with Byzantine faults

2. Implementing atomic broadcast
   - Sequencer-based approaches
   - Token-based approaches
   - Communication history-based approaches
   - Consensus-based approaches

3. Consensus
   - Definition
   - Transformations

Implementing atomic broadcast

Classification of well-known algorithms according to Défago et al.:

- Sequencer-based approaches
  - Selected node determines message order
  - Fixed or moving sequencer
- Token-based approaches (“privilege-based”)
  - Serialisation of all messages: only token owner may send
  - Active token / passive token
- “Communication history”
  - Sender assigns time stamps, receiver sorts messages
- Consensus-based approaches
  - Order determined by distributed consensus algorithms

Sequencer-based approaches

Sequencer defines message order

- Variant “UB”:
  - Sender sends message to sequencer (unicast)
  - Sequencer sends message to all nodes (broadcast)
  - High load / bottleneck at sequencer
- Variant “UUB”:
  - Sender sends message to sequencer (unicast)
  - Sequencer assigns sequence number to sender (unicast)
  - Sender sends message + sequence number to all nodes (broadcast)
  - Less work for sequencer, but higher latency
- Variant “BB”:
  - Sender sends message to all nodes (including sequencer; broadcast)
  - Sequencer assigns sequence number and sends it to all nodes (broadcast)

In all cases: What if sequencer crashes?

Additionally for UUB and BB: What if sender crashes?

Token-based approaches

- Only token owner may send messages
- Circulating token
  - Unused token is passed to next node automatically
  - Efficient if many nodes broadcast messages
  - Token circulates even if no one wants to broadcast messages
- Requesting the token on demand
  - Token is passed only upon request
  - Efficient if only one / few nodes want to broadcast
  - Many senders: High overhead for searching for the token

Lost token

- A crashed holder is indistinguishable from a slow holder
- Issuing a new token needs a failure detector or a timeout
- Again not feasible in asynchronous system

Communication history-based approaches

- Order is defined by the sender
- Messages usually carry a (physical or logical) timestamp
- Recipients observe messages generated by other processes and their timestamp
  - i.e., the “communication history”
- Process can deliver a message *m* as soon as it knows that there cannot be any other message that needs to be delivered before *m*

Communication history-based approaches

- Order is defined by the sender
- Messages usually carry a (physical or logical) timestamp
- Recipients observe messages generated by other processes and their timestamp
  - i.e., the “communication history”
- Process can deliver a message m as soon as it knows that there cannot be any other message that needs to be delivered before m

Simple examples:

- Synchronous system: Physical time stamps; delivery delayed by upper bound of clock error + transmission delay
- Asynchronous system: Round-robin approach: Deliver message 1 of $P_1, P_2, \ldots$, then message 2, etc.

Round-robin and crashes

- The synchronous rule works because the delay bound says when an earlier message can no longer arrive
- The asynchronous round-robin rule waits for message k from every process
- A crash looks like a process that has not sent yet
- So a correct process waits forever

Consensus-based approaches

Distributed consensus algorithms:

- Each node proposes some value
  In this case: next message ID / list of n message IDs
- Result of consensus: One of the proposed values
  Here: Next message / next n ordered messages

Next: transformation: abcast $\leftrightarrow$ consensus

Atomic broadcast and consensus

So far

- Simple solutions for bcast, fbcast, cbcast in asynchronous system
- Solution for uniform bcast if less than half of the processes are faulty
- No solution for abcast in asynchronous system

Next step:

- Atomic broadcast and the consensus problem

Consensus

Definition of the *consensus problem:*

Each process (in a set of n processes) proposes a value, and all correct processes decide upon a common value, such that the following properties are satisfied:

- *Termination:* Every correct process eventually decides some value
- *Agreement:* No two correct processes decide on different values v, v'
- *Validity:* If a correct process decides v, then v was previously proposed by some process.
- *Integrity:* No process decides twice

Note: some other definitions with minor differences exist!

Consensus variants

- *Uniform agreement:* If a process (whether correct or faulty) decides v, then all correct processes eventually decide v.
- *Uniform integrity:* If a process (whether correct or faulty) decides v, then v was previously proposed by some process.
- *Validity':* If all processes propose the same value v, then this is the only possible decision value
- *Validity":* If all correct processes propose the same value v, then this is the only possible decision value

Transforming atomic broadcast into consensus

- Algorithm for atomic broadcast also solves the consensus problem. . .
  - The transformation uses no timing assumption, so it works for any synchrony model

```
Uses: Atomic Broadcast (abcast)

upon Init:
   decided := false;

upon event < Propose, v > do:
   trigger < abcast.Broadcast, v >;

upon event < abcast.Deliver, u > do:
   if not decided then
      trigger < Decide, u >;
```

Which property is used

- **Termination:** A correct proposal is delivered, so the handler runs
- **Agreement:** Total order, so every correct process delivers the same first message
- **Validity:** Abcast integrity, so that message was proposed
- With reliable broadcast instead of atomic broadcast, the two processes can deliver different first messages and agreement fails

Transforming rbcast + consensus into abcast

```
Uses: Reliable Broadcast (rbcast), MultiConsensus (cons)

upon Init:
   R_delivered := ∅; A_delivered := ∅; k := 0

upon event < ABroadcast, m > do:
   trigger < rbcast.RBroadcast, m >;

upon event < rbcast.RDeliver, m > do:
   R_delivered := R_delivered ∪ {m}

repeat forever:
   A_undelivered := R_delivered - A_delivered
   if A_undelivered ≠ 0 then
      k := k + 1
      trigger < cons_k.Propose, A_undelivered >
      wait

upon event < cons_k.Decide, msgSet >:
   forall v in (msgSet - A_delivered) in deterministic order
      trigger < ADeliver, v >
   A_delivered := A_delivered ∪ msgSet
   resume wait
```

Summary

1. Reliable broadcast with Byzantine faults

2. Implementing atomic broadcast
   - Sequencer-based approaches
   - Token-based approaches
   - Communication history-based approaches
   - Consensus-based approaches

3. Consensus
   - Definition
   - Transformations

References I

Gabriel Bracha. An asynchronous $\lfloor (n-1)/3 \rfloor$-resilient consensus protocol. In *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 154–162, 1984. 10.1145/800222.806743.

Xavier Défago, André Schiper, and Péter Urbán. Total order broadcast and multicast algorithms: Taxonomy and survey. *ACM Computing Surveys*, 36(4):372–421, 2004.

Sam Toueg. Randomized Byzantine agreements. In *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 163–178, 1984. 10.1145/800222.806744.